

Configurable retail profiles without rebuilding the app

A fast, maintainable way to let each showroom choose its retail type and receive adapted UI copy, icons, graphics, product fields, calculators and AI tools - while preserving one React and Supabase codebase.

Recommended MVP decision

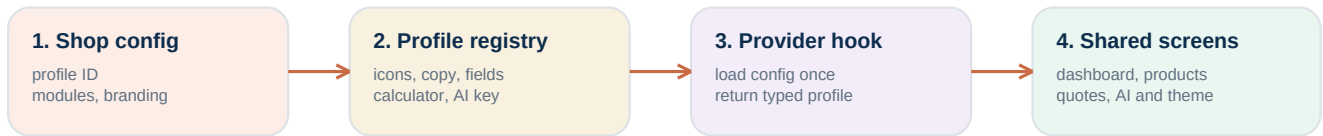
Store a profile ID and enabled operation modules on the shop. Keep the definition of each profile in a versioned TypeScript registry. Load it through one provider/hook, then let existing screens consume configuration. Do not build a visual form builder, separate app themes or dynamic database schema editor in version one.

The constraint that saves time

- The core navigation, customer identity, permissions, product ID, quote ID and order flow remain shared for all businesses.
- A profile may change language, icon, empty-state graphics, dashboard widgets, product attributes, calculator and AI prompt templates. It must not fork routes or duplicate components.
- A business can activate more than one operation module. For example, tiles uses variants plus project sales; electronics uses variants plus serialized products.

Target architecture: configuration over branching

Use one chain of responsibility. A signed-in shop provides settings, the static registry turns them into a typed profile, and screens render from that profile.



One codebase. One shared data model. Small configuration changes create different showroom experiences.

Why this is the lowest-risk approach

- Fast: profile definitions live in one source file, so a new showroom type is mostly data/configuration rather than a new feature branch.
- Safe: permissions, RLS, routes and customer records do not change with the visual profile.
- Testable: every profile can be tested against the same set of shared screens and core user journeys.
- Reversible: an owner can change profile/modules later; old records still render because their core fields remain unchanged.

Do not do in MVP

- Do not create a separate component tree, table set or theme package per industry.
- Do not let arbitrary client JSON define unsafe AI prompts, database access or permissions.
- Do not make every attribute a top-level products column. Keep universal fields structured and profile-specific display attributes contained.

Data model and migration plan

Add only the two configuration fields needed now. Keep the profile registry in source control so product behaviour is reviewable, typed and versioned.

Where	Add	Purpose
shops	retail_profile_id text	Primary vertical such as tiles, furniture, bathware, lighting, electronics, pharmacy, salon or standard_retail.
shops	enabled_modules text[]	Operation overlays such as variants, serialized, batch_expiry, project_sales, service, delivery, wholesale and multi_store.
shops	branding jsonb default '{}'	Optional logo URL, display name and safe theme preference. Keep sensitive/permission data out of this object.
products	attributes jsonb default '{}'	MVP display attributes such as finish, fabric, wattage or warranty. Keep name, SKU, price, stock and image as normal columns.

Migration rules

- Create a new timestamped migration. Do not edit the initial schema after it may have been deployed.
- Backfill existing shops as retail_profile_id = 'tiles' or use a temporary 'showroom' default, then show the profile selection on next sign-in.
- Add defaults and validation checks for supported IDs. Profile/module eligibility should be validated in the shared registry and server-side write path.
- Keep tenant RLS. Only the shop owner or approved role may update retail profile, modules and branding. Never allow a client to set another shop ID.
- Use attributes jsonb only for profile-specific metadata. Move high-volume filter/report fields to relational columns later, based on real usage.

Profile registry and provider contract

Make the static registry the single source of truth for supported profile behaviour. The database stores IDs, not a large free-form UI specification.

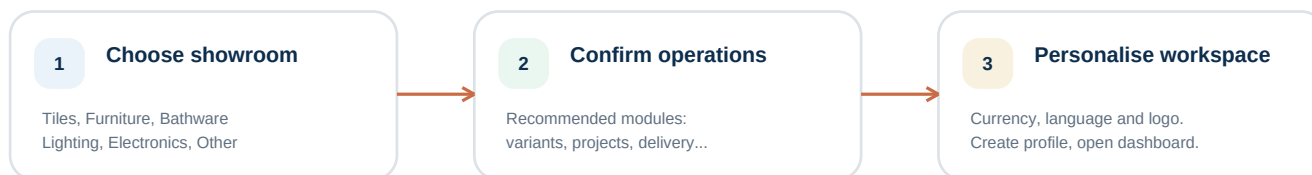
Registry property	Example	Consumed by
id and displayName	tiles / Tile showroom	Onboarding, header, settings and analytics labels.
recommendedModules	variants, project_sales, delivery	Onboarding defaults and enabled feature checks.
copy and icon	Project, room, sample, material icon	Dashboard, empty states, product/card labels.
productFieldSchema	size, finish, material, colour	Product form and product card secondary details.
calculatorKey	area_wastage	Quote builder calculation panel and validation.
aiProfileKey	interior_sales	Server-generated suggested actions and prompt template.

Provider behaviour

- Create RetailProfileProvider after authentication and fetch the authenticated profile/shop once. Resolve retail_profile_id through the local registry and merge enabled modules and safe branding.
- Expose useRetailProfile() returning { profile, modules, hasModule, labels, productFieldSchema, calculatorKey, aiProfileKey, loading, error }.
- Keep a safe fallback profile called showroom. Unknown IDs should not crash the application; log a non-PII diagnostic and use the fallback.
- Do not fetch the shop configuration individually in every page. Existing MobileLayout and pages should read from the provider.

Fast onboarding and profile switching

Ask one meaningful business question, preselect the right operating modules and let the owner confirm. The flow should be short enough to feel like personalisation, not setup work.



Keep this setup to three short screens. Users can later edit modules in Settings without losing existing data.

Recommended profile mappings

Profile	Recommended modules	Initial adaptation
Tiles / Bathware	variants, project_sales, delivery, installation	Area/space language, finish/size fields, room visualise and area calculator.
Furniture	variants, project_sales, delivery, installation	Fabric/dimension fields, delivery and assembly estimator.
Electronics	variants, serialized, warranty	Serial/warranty fields, product comparison and installation options.
Pharmacy / Grocery	batch_expiry, variants	Batch/expiry alerts, supplier replenishment and low-stock dashboard.
Salon / Tailoring	service, appointments	Bookings, service packages, customer history and reminders.

- Always permit the owner to edit the profile and modules later in Settings. Explain that disabling a module hides it but does not delete existing data.
- Profile changes should trigger a confirmation page describing what will change: labels, optional fields, dashboard widgets and quote calculator.
- Do not ask the user to select from the seven operation types first. Ask the business category first, then recommend a combination of modules.

Screen adaptation rules

Make the application recognisably the same for every showroom. Adapt context, not navigation architecture.

Screen	Shared structure	Profile adaptation
Top header	Logo, showroom name, language, notifications.	Small category icon, one-line workspace subtitle and selected brand assets.
Dashboard	Actions, four metrics, follow-up list.	Tiles: open quote area. Electronics: warranties. Pharmacy: expiry. Salon: appointments.
Customers	Pipeline, next action, owner, call/WhatsApp/quote.	Label Visit as consultation, booking or site measure when profile requires it.
Products	Search, availability, image, price, Add to quote.	Card secondary fields come from productFieldSchema; fields stay limited to three.
Quote	Customer, line items, taxes, total, send/share.	Calculator panel, units, delivery/install/service lines change by profile/modules.
AI	Assistant layout, chat and suggested prompts.	Safe profile-specific prompt suggestions and tool cards. AI data remains server-generated and minimised.

Time-saving UI rules

- Keep the current bottom navigation and shared page shells. Change labels only when the new label is more accurate, not merely different.
- Use existing Lucide icons and the existing neutral/navy/terracotta design system. Add a small profile graphic or material texture in heroes and empty states only.
- Use one ProductCard, one CustomerCard and one QuoteBuilder. Pass profile-driven props instead of copying components for tiles, furniture and electronics.
- Hide unavailable widgets/tools cleanly. Never show an icon, tab or form field that has no underlying workflow.

Product, quote and AI implementation details

Reuse common records and isolate the profile-specific logic behind small, typed adapters.

Product forms

- The common Product form always owns name, SKU, brand, category, price, price unit, stock status, image and description.
- Render additional fields from productFieldSchema. Validate values client-side for usability and validate/sanitise again in the server-side write path when business-critical.
- Store values under products.attributes for the first release. Do not query/filter deeply nested attributes across the whole catalogue until a real reporting need justifies extracting a field.

Quote calculators

- Define a calculator registry: generic, area_wastage, service_booking, delivery_installation. Each returns a normalised quote draft with line items, quantities, taxes and totals.
- The QuoteBuilder selects calculatorKey from the profile; it must never contain profile-specific arithmetic inline in the React page.
- Persist the calculator key and immutable item snapshots with each saved quote so historical totals remain explainable after profile changes.

AI

- The browser sends a user request and selected safe IDs. The authenticated Edge Function loads the shop profile and builds the allowed profile-specific instruction server-side.
- Examples: tiles suggests visualise a space and area estimation; electronics suggests compare products and warranty help; salon suggests appointment follow-up drafts.
- Do not send raw customer phone numbers or whole customer records to the AI provider. Keep consent, limits and rate-limits from the security remediation work.

Lowest-time delivery sequence

Work in small testable slices. Stop after phase 3 if you only need the profile-driven UI; phases 4 and 5 make product and AI behaviour richer.

Phase	Deliverable	Files most likely touched	Proof
1. Foundation	Migration, registry, provider, fallback profile.	new migration, retailProfiles.ts, RetailProfileProvider.tsx, App.tsx	Existing shop opens with safe default profile.
2. Onboarding	Three-step profile/module selection and owner settings update.	Onboarding.tsx, Settings/More route, provider tests	New shop saves profile/modules and returns to dashboard.
3. UI adaptation	Header, dashboard copy/widgets, product labels/fields and empty states use provider.	MobileLayout, Dashboard, Products, AddProductModal, shared cards	Tiles and electronics render different copy with same routes.
4. Quote adapter	Calculator registry and profile-driven quote sections.	QuoteBuilder, calculator modules, quote persistence/RPC	Generic and area_wastage totals match tests.
5. AI adapter	Server-side profile-specific suggestions and instruction builder.	gemini-proxy Edge Function, AI.tsx	No PII transfer; profile key changes suggestions safely.

Definition of done

- A user can select a business category, accept/edit recommended modules and see the saved profile on their next login.
- The same routes render safe and relevant labels/fields for at least tiles and electronics without duplicate page components.
- Changing a profile is owner-protected, tenant-safe and does not delete products, customers or quotations.
- Build, lint, migration tests and at least two profile integration tests pass.

1. Foundation and migration

Run this prompt as a self-contained Antigravity task after completing the preceding step. Review the resulting migration and tests before moving on.

ANTIGRAVITY PROMPT - 1. FOUNDATION AND MIGRATION

Implement the first RetailProfile foundation in the existing RetailFlow React and Supabase app. Do not create separate apps or duplicate pages. Create a new timestamped Supabase migration that adds `shops.retail_profile_id`, `shops.enabled_modules` `text[]` and `shops.branding` `jsonb` with safe defaults and validation. Preserve tenant RLS and ensure only an owner can change these configuration fields. Add `products.attributes` `jsonb` for profile-specific display metadata while retaining all existing common product columns. Create `src/config/retailProfiles.ts` as a typed static registry for tiles, furniture, bathware, lighting, electronics, pharmacy, salon and a showroom fallback. Each registry entry must define display copy, icon key, recommended modules, product field schema, calculator key and AI profile key. Add a `RetailProfileProvider` and `useRetailProfile` hook that fetch the authenticated shop once, resolves the registry and safely falls back for unknown profiles. Integrate it at the authenticated app root. Run build and lint; add focused tests for fallback, module checks and tenant-safe updates.

Review before accepting the change

- No duplicated industry page components or client-side bypass of tenant security.
- No operation module is visible without an implemented workflow or an explicit safe disabled state.
- Existing RetailFlow UI refresh, product/customer filters and quote layout remain intact.
- All changed code uses the shared profile provider and registry rather than hard-coded string checks scattered across pages.

2. Onboarding and settings

Run this prompt as a self-contained Antigravity task after completing the preceding step. Review the resulting migration and tests before moving on.

ANTIGRAVITY PROMPT - 2. ONBOARDING AND SETTINGS

Extend the existing RetailFlow onboarding into a fast three-step retail profile setup. Step 1 selects business category, not an operation type. Step 2 shows preselected operation modules that the owner can edit: variants, serialized, batch_expiry, project_sales, service, delivery, installation, warranty, wholesale and multi_store. Step 3 saves name, currency, language and optional safe branding. Save retail_profile_id, enabled_modules and branding only for the authenticated owner shop. Do not allow changing another shop ID. Add a working Settings/More entry where an owner can later change profile/modules; explain that disabling modules hides UI but never deletes existing data. Use current RetailFlow visual style, responsive mobile layout, accessible controls, loading/error states and no placeholder actions. Add tests for preselection, persistence, owner protection and editing an existing shop.

Review before accepting the change

- No duplicated industry page components or client-side bypass of tenant security.
- No operation module is visible without an implemented workflow or an explicit safe disabled state.
- Existing RetailFlow UI refresh, product/customer filters and quote layout remain intact.
- All changed code uses the shared profile provider and registry rather than hard-coded string checks scattered across pages.

3. Adapt shared UI without forking

Run this prompt as a self-contained Antigravity task after completing the preceding step. Review the resulting migration and tests before moving on.

ANTIGRAVITY PROMPT - 3. ADAPT SHARED UI WITHOUT FORKING

Make RetailFlow UI profile-aware using useRetailProfile without copying MobileLayout, Dashboard, Customers, Products, ProductCard, AddProductModal or QuoteBuilder. Keep the current navigation and visual design system. Adapt only contextual UI: header subtitle/icon, dashboard widget labels, empty-state copy, customer consultation/visit labels, product form extra fields and up to three product-card secondary attributes. Existing core fields and routes must remain stable. Tiles should show size/finish/material and project language. Electronics should show warranty/serial-oriented fields and comparison language. Pharmacy should show batch/expiry language only when its module is enabled. Hide unavailable widgets and controls completely. Preserve working customer/product filters. Add rendering tests proving two profiles use the same routes/components but display different relevant labels and field schemas.

Review before accepting the change

- No duplicated industry page components or client-side bypass of tenant security.
- No operation module is visible without an implemented workflow or an explicit safe disabled state.
- Existing RetailFlow UI refresh, product/customer filters and quote layout remain intact.
- All changed code uses the shared profile provider and registry rather than hard-coded string checks scattered across pages.

4. Profile-driven quote calculators

Run this prompt as a self-contained Antigravity task after completing the preceding step. Review the resulting migration and tests before moving on.

ANTIGRAVITY PROMPT - 4. PROFILE-DRIVEN QUOTE CALCULATORS

Refactor RetailFlow QuoteBuilder so profile-specific arithmetic is not written inline in the page. Create a typed calculator registry with generic, area_wastage, service_booking and delivery_installation calculators. Each calculator returns a normalised quote draft: line items, quantities, units, tax, discount, delivery, service/installation amount and total. QuoteBuilder selects calculatorKey from useRetailProfile. Tiles expose area, waste percent and boxes; furniture exposes delivery/assembly; services expose appointment/service lines; generic retail shows item/quantity/discount. Preserve the current quote UI quality. Persist calculator key and immutable item snapshots with saved quotes so historical totals remain explainable after a profile change. Add unit tests for all calculator edge cases and integration tests for generic and area_wastage quote totals.

Review before accepting the change

- No duplicated industry page components or client-side bypass of tenant security.
- No operation module is visible without an implemented workflow or an explicit safe disabled state.
- Existing RetailFlow UI refresh, product/customer filters and quote layout remain intact.
- All changed code uses the shared profile provider and registry rather than hard-coded string checks scattered across pages.

5. Secure profile-aware AI

Run this prompt as a self-contained Antigravity task after completing the preceding step. Review the resulting migration and tests before moving on.

ANTIGRAVITY PROMPT - 5. SECURE PROFILE-AWARE AI

Make RetailFlow AI profile-aware without trusting client-supplied system prompts. The React client may send the user message and safe selected record IDs only. In the authenticated gemini-proxy Edge Function, derive the caller user, shop and retail profile server-side, then resolve the profile AI key from a server-side allow-list. Build profile-specific suggested actions and instruction templates there: tiles can visualise spaces and explain area/waste; electronics can compare products and warranties; salons can draft appointment follow-ups. Do not send customer phone numbers, full notes or raw customer lists to Gemini. Preserve JWT verification, tenant checks, request validation, image limits and rate limits. Add tests proving a caller cannot select another shop profile or inject a raw system instruction, and that phone numbers are absent from normal AI payloads.

Review before accepting the change

- No duplicated industry page components or client-side bypass of tenant security.
- No operation module is visible without an implemented workflow or an explicit safe disabled state.
- Existing RetailFlow UI refresh, product/customer filters and quote layout remain intact.
- All changed code uses the shared profile provider and registry rather than hard-coded string checks scattered across pages.

6. Verification and rollout

Run this prompt as a self-contained Antigravity task after completing the preceding step. Review the resulting migration and tests before moving on.

ANTIGRAVITY PROMPT - 6. VERIFICATION AND ROLLOUT

Add a focused RetailProfile test plan and rollout guardrails to RetailFlow. Cover migration defaults, owner-only profile changes, RLS tenant isolation, unknown profile fallback, onboarding recommendations, profile persistence after sign-in, UI adaptation for tiles and electronics, conditional modules, calculator selection and profile-aware AI safety. Add browser-level checks for mobile layout so the profile header and bottom navigation do not overlap content. Run build, lint and test suite. Produce a concise implementation summary listing changed files, migration name, supported profile IDs, module IDs, any unfinished integrations and exact manual staging steps. Do not modify unrelated UI, remove working filters or weaken existing security controls.

Review before accepting the change

- No duplicated industry page components or client-side bypass of tenant security.
- No operation module is visible without an implemented workflow or an explicit safe disabled state.
- Existing RetailFlow UI refresh, product/customer filters and quote layout remain intact.
- All changed code uses the shared profile provider and registry rather than hard-coded string checks scattered across pages.